

Procesrapport

Automatisk styring af affugter

Contents

Indledning	2
Planlægning og arbejdsproces.....	2
Prototype	2
Arbejdsproces	3
Valg af løsning og teknologi	4
Hardware	4
Software	4
API	4
Web	5
Udviklingsprocessen	5
Review	6
Problemer og problemløsning.....	6
Test og kvalitetssikring	6
Resultat og evaluering	7
Proceslog	7

Indledning

Denne procesrapport beskriver udviklingsforløbet for projektet omkring udvikling af et automatisk styret affugtersystem. Systemet er udviklet med henblik på automatisk at styre en affugter på baggrund af den aktuelle luftfugtighed og elpris.

Rapporten fokuserer på selve arbejds- og udviklingsprocessen, herunder planlægning, arbejdsfordeling, tekniske valg, udfordringer undervejs samt test og evaluering af processen. En nærmere beskrivelse af systemets funktionalitet, opbygning og endelige løsning findes i produktrapporten.

Planlægning og arbejdsproces

Udviklingen af dette produkt er foregået i to faser; prototype og udforskning af teknologier, og endelig udvikling.

Prototype

Prototypefasen gik på at udforske vores muligheder fra hardware til webframework.

Her har vi testet forskellige boards; Arduino R4 Wi-Fi, Raspberry Pi Pico W, og Raspberry Pi 4 og sensorer; DHT11, DHT22 og en selvstændig Zigbee Bluetooth luftfugtighedssensor, for at se hvor langt løsningen kunne koges ned, samtidig med at vi bevarer den nødvendige funktionalitet. Pico W og Arduino-boardene var mere end tilstrækkelige til dataindsamling, både elpris- og sensordata, men ville af nødvendighed skulle kommunikere de data til en ekstern server. Vi vurderede i dette tilfælde, at det ville være mere brugervenligt at eliminere det ekstra tilslutningsskridt, og opstille en samlet løsning - derfor blev Raspberry Pi 4 den endelige løsning.

Vi har også testet forskellige API'er; Elprisenligenu, Billigkwh, DanishGrid, og endte med Energidaservice for at opnå den højeste opløsning i elprisdata. De fleste offentligt eksponerede API'er udstiller kun data ned til 1 elpris pr. time, eks. Elprisenligenu og Billigkwh, hvor andre, lidt mindre tilgængelige API'er, udstiller ned til elseskab, leverandør, og adresse – og med datapunkter ned til kvarteret. DanishGrid var i første omgang valget, men grundet en længere outage fra deres side, valgte vi at skifte endeligt til Energidaservice, da de udstiller data af samme granularitet, men har vist sig at have en bedre opetid.

Da vi har arbejdet med flere ukendte, har processen været agil og iterativ. Det har været nødvendigt ved de fleste skridt i processen, at foretage en fælles vurdering af hvert komponent i løsningen, for at holde scopet stramt. Dette har foregået ved løbende gennemgang og vurdering af hverandres arbejde, og løbende skøn af hvor meget tid, en opgave vil tage.

Tidsskønnet er gjort på baggrund af erfaring - eller manglende erfaring - med et emne eller en teknologi. Det kan aldrig gøres helt præcist, men der sættes tid af til research og test. Hvis denne trækker ud, lægges opgaven i backloggen.

Prototype- og planlægningsprocessen har bestået af daglige standupmøder, mange gange flere møder.

Vi har ikke afholdt stringente sprint. Dette skyldes en blanding af faktorer; navnlig projektets stærkt indskrænkede scope, og størrelsen på opgaver. Det har aldrig været nødvendigt at sætte et helt f.eks. ugentligt sprint af til en række opgaver, da hver opgave er stærkt begrænset, og gennemgås under fælles review.

Arbejdsproces

Standups er af princip afholdt dagligt. Dette sørger for, at begge udviklere altid er klar over hvad den anden laver, og at der aldrig hersker tvivl om, hvad der skal gøres af arbejde.

Opgavestyringen er ført over vores fælles Trello-board. Det er et redskab vi har erfaring med, og synes skaber et godt overblik over opgaver, kan let vedligeholdes og opdateres, og giver en komplet audit log over projektets forløb.

Trello-boardet har bestået af følgende kolonner:

- **Projektafklaring;**

Hovedsageligt brugt i projektets prototypestadiet. Opgaver i denne kolonne har haft til formål at definere krav til projektet, valg af teknologi, metodevalg, rollefordeling o.l.

- **Dokumentation**

Alle dokumentationsopgaver oprinder her. Diagrammer, beskrivelser, problemformulering o.l.

- **Udvikling**

Til konkrete udviklingsopgaver, f.eks. integration af fugtighedssensor via Raspberry Pi 4-boardets GPIO pins, databaseoprettelse, elprisdataindhentning.

- **In progress**

Når en opgave aktivt arbejdes på, bliver den tildelt til en udvikler og flyttet over i 'In progress', så vi altid har overblik over hvem der arbejder på hvad og hvornår.

- **Review**

Når en opgave i første omgang er færdig, flyttes den til at starte med over i 'Review'-kolonnen, hvor den bliver gennemgået, inden den anses for at være færdig. Det involverer altid begge udviklere på projektet, og i visse tilfælde godkendelse fra ekstern. I teorien fra forretningen, i praksis fra vejleder.

- **Done**

Når en opgave er gennemgået og godkendt, flyttes den over i 'Done'-kolonnen.

- **Backlog/nice-to-have**

Opgaver som vurderes til enten at være store, udenfor vores umiddelbare kompetencer, eller umiddelbart udenfor scope, bliver lagt i backloggen. Vi kan på den måde holde fast på fremtidige ønsker og idéer, til et punkt i processen, hvor der er mere luft til at udforske større tilføjelser. Konkret var Dashboardet og brugerstyringssiden i første omgang i backloggen, og blev implementeret, da vi senere i projektets løb havde tid til at implementere dem.

Valg af løsning og teknologi

Hardware

Fundamentalt var idéen, at systemet skulle bestå af en mikrocontroller, et relæ, og en sensor. Vi har eksperimenteret med Arduino R4 Wi-fi boards samt Raspberry Pi Pico boards, og endte til sidst på en lidt mere kraftfuld mikrocomputer – Raspberry Pi 4. Beslutningen om at gå videre med så stærk en computer som den, byggede på følgende:

Ligesom Arduino og Pico boards kan Raspberry Pi 4 indhente elprisdata fra API samt indsamle sensordata via boardets egne GPIO pins. Den endelige beslutning om at bruge Raspberry Pi 4 gik på, at maskinen er stærk nok til at hoste egen database og webserver.

Et mindre kraftfuldt board som en Arduino R4 Wi-fi ville kunne opsamle og videresende data til en ekstern server, men vi vurderede at dette ville skabe et unødvendigt ekstra led i processen.

Fremfor en løsning med et fritstående relæ, som ville skulle loddess til system-boardet, valgte vi i stedet en 'smart plug'. Det er en stikprop, som opretter forbindelse til et lokalt netværk, og kan styres over netværket med en række endpoints. Med et konventionelt relæ, ville affugter, Raspberry Pi og relæ skulle loddess sammen til en enkelt enhed. En stikprop med indbygget, fjernstyret relæ, giver os en betydeligt mere modulær opstilling, og det ville evt. være muligt i fremtiden af skifte til en anden maskine, uden at skulle afmontere den eksisterende løsning.

Software

Den centrale applikation er skrevet i Python. Der findes uendeligt mange muligheder for udviklingen af et sådant system, men Python er valgt ud fra erfaring, samt det brede udvalg af biblioteker, som gør at man let kan opstille lightweight-løsninger. En tungere web- og API-applikation i et mere full-featured framework, som f.eks. .NET ville have tærbetydeligt mere på tilgængelige ressourcer – særligt tidligt i processen, hvor Raspberry Pi ikke var endegyldigt valgt.

API

Den centrale applikation er bygget på biblioteket 'FastAPI'. Dette faciliterer bygningen af en række mindre routes, som modtager, behandler og sender data gennem systemet.

Denne modtager sensor- og elprisdata, behandler dem, og styrer smart-plug'en derudfra. Der findes i API'et også en længere række endpoints til debugging og manuel styring. Oversigt over disse findes som bilag i produktrapporten.

API'et er udviklet gennemgående asynkront. Det er ikke som sådan en nødvendighed, da systemet er tiltænkt en enkelt sensor. Det gør dog, at systemet let kunne udvides til flere sensorer, flere målinger og flere styrede enheder.

Web

Hele løsningen er hostet via biblioteket 'Uvicorn'. Dette er en webserver, som udstiller hele løsningen lokalt på et netværk. Indenfor projektets scope kan serveren altså ikke tilgås udefra, da den ikke er offentligt eksponeret. Det ville være muligt, hvis brugeren ønsker fjernstyring uden for hjemmet, at tilkoble serveren til et privat VPN-netværk som Tailscale og benytte den manuelle brugerstyringsside. Løsningen er dog designet til at køre uden user intervention.

Denne løsning er også valgt ud fra tidligere erfaring. Vi har hver især arbejdet med lignende frameworks i skole- og arbejdsregi. Af samme årsag har vi valgt et framework, som vi har erfaring med, så vi har kunnet fokusere mere af vores indsats på dele af projektet, som vi har mindre erfaring med, f.eks. projektstyring og produkt- og procesrapporter.

Udviklingsprocessen

Processen har været meget iterativ og præget af samarbejde. Vi har hver taget vores opgaver, men ingen opgave er løst uden at den er blevet gennemgået og reviewet af den anden udvikler. Metoden er agilt inspireret, men holder sig ikke stringent til de agile principper. Flere faktorer har spillet ind:

- **Projektets tidsramme;**

Vi har haft begrænset tid til projektet. Vi kunne have afholdt sprint à eks. 1-2 uger, men der har ikke været en 'rød tråd' i et opgaveforløb henover så lang tid. Vi har afgrænset opgaverne skarpt, og arbejdet på udvikling og dokumentation sideløbende.

Et alternativ kunne have været et 2-ugers fokuseret sprint på udvikling, og et 2-ugers fokuseret sprint på dokumentation. Det er dog vores oplevelse og erfaring, at udvikling og dokumentation går hånd i hånd og komplementerer hinanden, og har ikke fundet værdi i at opdele dem.

- **Projektets scope;**

I et større projekt, hvor teknologi og proces var kendt på forhånd, havde det givet mening at opdele mere strikse udviklingsforløb, da der ville være et klarere overblik over teknologier og krav. Dette projekt har også haft et skarpt fokus på læring af projektstyring, samt prototypeudvikling og udforskning af for os ukendte teknologier.

- **Antallet af udviklere på projektet.**

På et større projekt med mange udviklere, ville det have givet mening, at opdele projektet i 'siloe', så hver udvikler sidder med sin del af projektet. Idet at vi er to, er hver opgave i større grad blevet til et mini-sprint, hvor opgaven er defineret, udført, reviewet og evalueret hver for sig.

Vi har flere gange udforsket løsninger hver for sig, hvor vi på hver vores måde har forsøgt at løse en opgave, reviewet produktet, og besluttet hvilket var mere passende til projektet.

Eksempler kunne være hardwarespørgsmålet. Her har vi hver især arbejdet med Arduino og Raspberry Pi respektivt, og endte med at vælge Raspberry Pi, da den tillod os yderligere

funktionalitet, f.eks. database og webserver over Arduinoen, som i dette tilfælde have uoverkommelige hukommelsesbegrænsninger.

Samme proces med at vælge API, hvor vi hver især har udarbejdet en løsning, og evalueret sammen.

Review

Når en opgave er løst, bliver den gennemgået af begge udviklere. Det tager naturligvis tid væk fra udvikling- og dokumentationsarbejde, men en fælles forståelse af produktet gør, at begge udviklere kan arbejde i flæng. Dette har været en prioritering, da vi er et lille team på to udviklere. Når hver udvikler har en dyb forståelse for produktet, reduceres risikoen for et totalt sammenbrud af processen ved f.eks. sygdom.

Problemer og problemløsning

Vores problemløsning er lignende iterativ. Problemer kan gemmes til følgende dags standupmøde, hvor vi sammen diskuterer en løsning og slår en opgave op på det pågældende problem. Vi har opgaver nok til at et enkelt problem ikke har været en hæmsko for vores udviklingsproces.

Vi har oplevet problemer som i systemets drift er relativt kritiske, f.eks. en utilgængelig smart plug, eller et API, som går ned.

I begge tilfælde er problemet taget op på standup. Vi har vurderet hvor kritisk problemet er, og hvor højt løsningen af det skal prioriteres. Af ovennævnte problemer tog API-problemet prioriteten, da det ikke kunne fastslås hvornår data igen ville være tilgængelige, og da elprisdata er en kritisk komponent i systemet, var det vigtigt at finde en alternativ løsning. Dagens andre opgaver blev derfor skudt tilbage af samme årsag, og genoptaget efter problemet var løst og reviewet.

Test og kvalitetssikring

I et system med flere fysiske dele, kan det ikke klares med unit tests og mock-data. Det har været nødvendigt for udviklingen at sikre, at disse fysiske dele – smart plug, sensor, Raspberry Pi – fungerer sammen i praksis.

Til formålet er systemet udstyret med en lang række debugging endpoints, som kan servere custom data til programmet, eksempelvis specifikke luftfugtighedsværdier eller elpriser, for at verificere at systemet agerer hensigtsmæssigt i respons til de værdier.

Vi har for lethedens skyld udviklet et debug dashboard, hvor man let og hurtigt kan indsende værdier, i stedet for manuelt at kalde et endpoint.

Systemet har overordnet set været aktivt kørende i al den tid det har været muligt. Dette har givet os mulighed for at føre overvågning og hypercare ved hver større udvikling. Næsten alle

systemets events logges både i terminal og i database – hver gang smart pluggens status bliver evalueret, hver gang der læses en luftfugtighedsværdi, og hver gang der sker en fejl.

Vi har derfor en lang, løbende oversigt over driftdata fra et virkeligt scenarie. Dette har ladet os fange fejl, som f.eks. et utilgængeligt API, eller en løs sensor. Dette kræver en struktur af stringent fejlhåndtering og logning på alle systemets handlinger og events.

Resultat og evaluering

Resultatet af vores arbejdsproces, arbejdsfordeling og fælles gennemgang er et system, som kan betydeligt mere end vi havde regnet med ved projektets start.

Nogle features har vist sig stadig være for store til en første iteration. Vi havde eksempelvis overvejet en lille machine learning model, som dynamisk kunne bestemme en mere gunstig pristærskel ud fra historiske data. Vi har gennem projektet haft mere basale opgaver at implementere, og denne ligger nu som en evt. fremtidig udvidelse og forbedring.

Andre features kom i backloggen fra starten af – eksempelvis brugervenlige dashboards med adgang til historiske og aktuelle data og debug endpoints, eller implementeringen af mulighed for flere datakilder, eks. en luftfugtighedssensor, som kan tilkobles via Bluetooth/netværk, i stedet for fysisk at skulle sættes til Raspberry Pi'ens GPIO.

Vi har også implementeret dele, som vi har været nødt til at skære væk, da det efter evaluering har vist sig ikke at være den bedste løsning på opgaven. Vi forsøgte os med at køre applikationen i Docker, så vi let kunne opsætte samme applikation på flere maskiner. Dette voldte dog en del problemer, da en dockerised løsning har mange hurdles ved at indhente API-data, og med at kontakte GPIO-pins på Raspberry Pi. Vi var tilfredse med arbejdet, men løsningen passede ikke til problemet.

Proceslog

Som bilag til procesrapporten vedlægges en proceslog baseret på projektets aktivitetshistorik fra Trello. Procesloggen giver et kronologisk overblik over projektets forløb og dokumenterer blandt andet oprettelse og afslutning af opgaver, ændringer i opgavernes status, kommentarer, tekniske beslutninger og ændringer foretaget undervejs.

Procesloggen er ikke tænkt som en detaljeret beskrivelse af selve løsningen, men som dokumentation for den faktiske arbejdsproces. Den kan derfor bruges til at følge projektets udvikling fra de indledende opgaver og afklaringer til implementering, test, ændringer og den afsluttende dokumentation.

Loggen er bearbejdet fra Trellos aktivitetshistorik, så interne og tekniske Trello-hændelser uden betydning for projektforsløbet er sorteret fra, mens relevante hændelser er bevaret og præsenteret kronologisk i et mere læsbart format.